

The ADMM Norms: M , N , and the Adjoint of A

Optimal Transport / Schrödinger Bridge Project
Companion to `nonuniform_grid_xy_variables.tex` (Steps 1–5)

Abstract

Continues `nonuniform_grid_xy_variables.tex`, which pinned down (Steps 1–5, confirmed) the two grids, f_1, f_2 , and the linear consensus map $A(q, b) = (\bar{\mathbf{I}}_t q, \bar{\mathbf{I}}_x b)$, and left one thing open: the inner products $\langle \cdot, \cdot \rangle_{\mathcal{X}}, \langle \cdot, \cdot \rangle_{\mathcal{Y}}$ (that document’s $\mathcal{X}, \mathcal{Y}$; this note calls their matrices N, M respectively, to match ongoing discussion). This note (a) writes down M, N concretely – the quadrature-consistent Riemann-sum norms, already sitting in `pipeline_nu.py` as a *diagnostic*, here promoted to the actual ADMM metric; (b) derives A^* , the N, M -weighted adjoint of A , in closed form; (c) proves $\|A^* A\|_N \leq 1$ *exactly*, for any grid in this family, giving an unconditional (not grid-tuned) stability bound $\tau > \gamma$ for §9’s linearized-ADMM iteration.

A companion draft, `norms_in_ladmm.tex` (older notation: half-integer indices, $\mathcal{A}, \tilde{\rho}$), independently derives the same weighted-average formula for the time-averaging adjoint (its Step 5b) via direct coefficient-collection rather than the general adjoint formula used here – a useful cross-check, reached two different ways.

1 Setup: M , N , and A , restated

Notation and indices exactly as in `nonuniform_grid_xy_variables.tex`: $q \in \mathbb{R}^{(n_t-1)n_x}$ at (t_j, ξ_k) , $j = 1, \dots, n_t - 1$; $b \in \mathbb{R}^{n_t(n_x-1)}$ at (τ_i, x_k) , $i = 0, \dots, n_t - 1$; $\rho, m \in \mathbb{R}^{n_t n_x}$ at (τ_i, ξ_k) (its ambient-space equation, §1). $\Delta t_i := t_{i+1} - t_i$ ($i = 0, \dots, n_t - 1$, the primal/cell widths); $\Delta t_j^{\text{dual}} := \frac{1}{2}(\Delta t_{j-1} + \Delta t_j)$ ($j = 1, \dots, n_t - 1$, the dual-mesh cell widths – §8.3’s $r_t/\bar{\mathbf{I}}_t$ construction, and the “dual grid” discussion from the same thread).

M (collocated/ y -grid norm):

$$\|(\rho, m)\|_M^2 = \Delta x \sum_{i,k} \Delta t_i \rho_{i,k}^2 + \Delta x \sum_{i,k} \Delta t_i m_{i,k}^2, \quad M = \begin{pmatrix} \text{diag}(\Delta t_i) \otimes \Delta x I_{n_x} & 0 \\ 0 & \text{diag}(\Delta t_i) \otimes \Delta x I_{n_x} \end{pmatrix}. \quad (1)$$

N (staggered/ x -grid norm):

$$\|(q, b)\|_N^2 = \Delta x \sum_{j,k} \Delta t_j^{\text{dual}} q_{j,k}^2 + \Delta x \sum_{i,k} \Delta t_i b_{i,k}^2, \quad N = \begin{pmatrix} \text{diag}(\Delta t_j^{\text{dual}}) \otimes \Delta x I_{n_x} & 0 \\ 0 & \text{diag}(\Delta t_i) \otimes \Delta x I_{n_x-1} \end{pmatrix}. \quad (2)$$

Both are literally `pipeline_nu.py`’s `norm_fn/dual_norm_fn (_weighted_sq_sum)` – currently used only to *measure* convergence, not to *run* the iteration. This note’s point is to promote them to the actual metric.

A (consensus map, linear part): $A(q, b) = (\bar{\mathbf{I}}_t q, \bar{\mathbf{I}}_x b)$, where (`nonuniform_grid_xy_variables.tex` §8.3, §10)

$$(\bar{\mathbf{I}}_t q)_i = \frac{1}{2}(q_i + q_{i+1}), \quad i = 0, \dots, n_t - 1 \quad (q_0 := q_{n_t} := 0), \quad (\bar{\mathbf{I}}_x b)_k = \frac{1}{2}(b_{k-1} + b_k), \quad k = 1, \dots, n_x \quad (b_0 := b_{n_x} := 0), \quad (3)$$

each acting independently per space-column ($\bar{\mathbf{I}}_t$) or per time-row ($\bar{\mathbf{I}}_x$) – $q_0, q_{n_t}, b_0, b_{n_x}$ set to 0 here since this is the *linear* part only (the affine boundary contribution is A ’s companion, d , not part of A itself).

2 The weighted adjoint, in general

$A^* : \mathcal{Y} \rightarrow \mathcal{X}$ is defined by $\langle Ax, v \rangle_M = \langle x, A^*v \rangle_N$ for all x, v . Writing A ’s plain (array-index) matrix as A too (abuse of notation): $\langle Ax, v \rangle_M = x^T A^T M v \stackrel{!}{=} x^T N A^* v$ for all x , so

$$A^* = N^{-1} A^T M. \quad (4)$$

N is SPD (diagonal, strictly positive entries), hence invertible – always well-defined.

3 Reduction to two independent 1D problems

A has no cross terms between $(q \rightarrow \rho)$ and $(b \rightarrow m)$, and M, N respect this same split – so A^* splits the same way, $A^*(\rho, m) = (A_t^* \rho, A_x^* m)$. Within each block, the averaging operator acts independently across many parallel “copies” (n_x space-columns for $\bar{\mathbf{I}}_t$, n_t time-rows for $\bar{\mathbf{I}}_x$), and in both cases the weight on the axis *not* being acted on is identical on the domain and codomain side (space’s Δx for the $q \rightarrow \rho$ block; time’s Δt_i , row by row, for the $b \rightarrow m$ block). The following lemma says that’s exactly the condition under which the many copies don’t interact – the full weighted adjoint (and its norm) reduces to the *single* 1D averaging matrix’s.

Lemma 1 (Parallel copies don’t change the weighted operator norm). *Let $P : \mathbb{R}^p \rightarrow \mathbb{R}^q$ be linear, D_1, D_2 SPD on $\mathbb{R}^p, \mathbb{R}^q$. For r independent copies of P (i.e. $P \otimes I_r$ on $\mathbb{R}^{pr}$), weighted by $D_1 \otimes C$ on the domain and $D_2 \otimes C$ on the codomain, $C := \text{diag}(c_1, \dots, c_r)$ **the same per-copy scalars on both sides**:*

$$\|P \otimes I_r\|_{D_1 \otimes C \rightarrow D_2 \otimes C} = \|P\|_{D_1 \rightarrow D_2}.$$

Proof. The squared norm is the top generalized eigenvalue of $(D_1 \otimes C)^{-1} (P \otimes I_r)^T (D_2 \otimes C) (P \otimes I_r) = (D_1^{-1} \otimes C^{-1}) (P^T D_2 P \otimes C) = D_1^{-1} P^T D_2 P \otimes I_r$ (the $C^{-1}C = I_r$ cancellation is exactly where “same scalars on both sides” is used), whose eigenvalues are those of $D_1^{-1} P^T D_2 P$, each with multiplicity r . $\square$

Applying Lemma 1: the $q \rightarrow \rho$ block reduces to $\bar{\mathbf{I}}_t : (\mathbb{R}^{n_t-1}, N_t) \rightarrow (\mathbb{R}^{n_t}, M_t)$ with $N_t = \text{diag}(\Delta t_j^{\text{dual}})$, $M_t = \text{diag}(\Delta t_i)$ ($c_k = \Delta x$, same for every space column k); the $b \rightarrow m$ block reduces to $\bar{\mathbf{I}}_x : (\mathbb{R}^{n_x-1}, \Delta x I) \rightarrow (\mathbb{R}^{n_x}, \Delta x I)$ ($c_i = \Delta t_i$, same for every time row i , since b, m share the identical time weight at each row).

4 A_t^* , explicitly

$A_t^* = N_t^{-1} \bar{\mathbf{I}}_t^T M_t$. Column j of $\bar{\mathbf{I}}_t$ is nonzero (value $\frac{1}{2}$) in rows $j-1, j$ only, so

$$(A_t^* v)_j = \frac{1}{\Delta t_j^{\text{dual}}} \left[\frac{1}{2} \Delta t_{j-1} v_{j-1} + \frac{1}{2} \Delta t_j v_j \right] = \frac{\Delta t_{j-1} v_{j-1} + \Delta t_j v_j}{\Delta t_{j-1} + \Delta t_j}, \quad j = 1, \dots, n_t - 1. \quad (5)$$

A *width-weighted* average of v ’s two flanking cells – not the plain $\frac{1}{2}, \frac{1}{2}$ average. It reduces to the plain average exactly when $\Delta t_{j-1} = \Delta t_j$ (locally uniform), and only then – exactly the

condition under which `nonuniform_grid_refine_ends.tex`’s “`interp_t_at_rho`: an accuracy caveat” section found the plain average’s error vanishing. This is not a coincidence: (5) is *forced* by M, N once A^* is required to be their genuine adjoint (Fact 1 below makes this precise), the same “structural, not approximate” character `nonuniform_grid_refine_ends.tex` §5 already claimed for the *Euclidean* adjoint on the uniform grid.

Fact 1. *No single scalar c satisfies $A_t^* = c \cdot \bar{\mathbf{I}}_t^T$. $\bar{\mathbf{I}}_t^T$ weighs $v_{j-1} : v_j$ in the fixed ratio 1 : 1 at every j ; A_t^* weighs them $\Delta t_{j-1} : \Delta t_j$, a ratio that changes from edge to edge on a graded grid. A fixed 1 : 1 ratio scaled by any single c cannot match a ratio that varies with j .*

Against the code

`operators_nu.py`’s `interp_t_at_rho` – called by `pipeline_nu.py`’s `At_fn`, which builds the gradient $\mathbf{g}$ *every* LADMM iteration, not just inside the projection – is

```
def interp_t_at_rho(in_arr):
    return 0.5 * (in_arr[:-1] + in_arr[1:])
```

the plain average, i.e. the Euclidean transpose $\bar{\mathbf{I}}_t^T$, not (5). Fact 1: this is not off by a rescaling, it is a genuinely different operator on any grid where consecutive Δt_i differ – which, for Chebyshev clustering, is *every* interior edge (unlike `refine_ends_time_grid`’s two isolated junctions).

5 A_x^* , explicitly

Same computation with Δx constant on both sides – it cancels entirely:

$$A_x^* = \bar{\mathbf{I}}_x^T, \quad (A_x^* m)_k = \frac{1}{2}(m_k + m_{k+1}). \quad (6)$$

Identical to `operators_nu.py`’s `interp_x_at_m_adj` (via `interp_x_at_phi` at zero BC) as coded today – **no change needed here**. Consistent with every earlier finding in this project: space, being uniform, never needed correcting; only the time-direction adjoint does.

6 Bounding $\|A^* A\|_N$

Fact 2. *For any bounded linear A between real inner-product spaces, $\|A^* A\|_N = \|A\|_{\mathcal{X} \rightarrow \mathcal{Y}}^2$.*

Proof. $\langle A^* A x, x \rangle_N = \|A x\|_M^2 \leq \|A\|^2 \|x\|_N^2$; $A^* A$ is self-adjoint and PSD w.r.t. $\langle \cdot, \cdot \rangle_N$, so its operator norm is its top Rayleigh quotient: $\|A^* A\|_N = \sup_x \langle A^* A x, x \rangle_N / \|x\|_N^2 = \sup_x \|A x\|_M^2 / \|x\|_N^2 = \|A\|^2$. $\square$

A is block-diagonal across $(q, b) \rightarrow (\rho, m)$, and M, N split the same way, so $\|A\|_{\mathcal{X} \rightarrow \mathcal{Y}} = \max(\|\bar{\mathbf{I}}_t\|_{N_t \rightarrow M_t}, \|\bar{\mathbf{I}}_x\|_{\Delta x I \rightarrow \Delta x I})$ (a Pythagorean argument: $\|A x\|_M^2 = \|\bar{\mathbf{I}}_t q\|_{M_t}^2 + \|\bar{\mathbf{I}}_x b\|_{M_x}^2 \leq \max(\cdot)^2 (\|q\|_{N_t}^2 + \|b\|_{N_x}^2) = \max(\cdot)^2 \|x\|_N^2$, tight by concentrating x on whichever block attains the max).

Lemma 2 (Both blocks are 1-Lipschitz). $\|\bar{\mathbf{I}}_t\|_{N_t \rightarrow M_t} \leq 1$ and $\|\bar{\mathbf{I}}_x\|_{\Delta x I \rightarrow \Delta x I} \leq 1$, for any $\{\Delta t_i\}$ (uniform or not) and any Δx .

Proof. Write $u_j := q_j$ ($j = 1, \dots, n_t - 1$, $u_0 := u_{n_t} := 0$ – the linear-part convention of §1), $v_i := (\bar{\mathbf{I}}_t q)_i = \frac{1}{2}(u_i + u_{i+1})$. By Cauchy–Schwarz, $v_i^2 \leq \frac{1}{2}(u_i^2 + u_{i+1}^2)$, so

$$\sum_{i=0}^{n_t-1} \Delta t_i v_i^2 \leq \frac{1}{2} \sum_i \Delta t_i (u_i^2 + u_{i+1}^2) = \frac{1}{2} \sum_{j=1}^{n_t-1} (\Delta t_{j-1} + \Delta t_j) u_j^2 = \sum_{j=1}^{n_t-1} \Delta t_j^{\text{dual}} u_j^2,$$

i.e. $\|\bar{\mathbf{I}}_t q\|_{M_t}^2 \leq \|q\|_{N_t}^2$ – using only $\Delta t_j^{\text{dual}} := \frac{1}{2}(\Delta t_{j-1} + \Delta t_j)$ ’s definition, nothing about uniformity. The $\bar{\mathbf{I}}_x$ case is the same argument with every weight equal to Δx (the special case $\Delta t_{j-1} = \Delta t_j = \Delta x$ throughout). $\square$

Essentially tight, not just a safe bound: a constant $q \equiv c$ pushes the ratio toward 1 as boundary effects wash out.

$$\boxed{\|A^* A\|_N \leq 1, \quad \text{for any } n_t, n_x, \text{ any clustering.}} \quad (7)$$

7 Consequence: an unconditional stability bound

`nonuniform_grid_xy_variables.tex` §9’s linearized-ADMM iteration needs $\tau > \gamma \|A\|_{\mathcal{X} \rightarrow \mathcal{Y}}^2$ for convergence (Fang–He–Liu–Yuan 2015). By (7), $\|A\|^2 \leq 1$ *exactly*, on every grid in this family – so $\tau > \gamma$ **suffices unconditionally**: no per-grid retuning, no asymptotic argument, a genuine equality-free bound holding at every finite n_t, n_x and every clustering strength. This is the first piece of the “grid-independent ADMM” goal delivered as a proof rather than an aspiration – contingent, of course, on actually switching the iteration’s A^* (§4–§5) and the projection’s adjoint machinery over to M, N , which is not yet done.

8 Empirical note: why R^* can’t wait

Wiring A_t^* into `pipeline_nu.py`’s `At_fn` alone – leaving `projection_nu.py`’s internal adjoint on the Euclidean metric – was tried and reverted: it satisfies the adjoint identity (§4, verified) and reduces to the old code on a uniform grid, but on an actual clustered grid (`sb.gaussian_nonuniform.py`, $n_t = n_x = 64$) it made the LADMM iteration diverge (residuals $\sim 10^{20}$), even though the projection still exactly enforced $R = 0$ throughout. Simplifying the y -update alone (`xy_variables.tex` §11.5, eq. 24) is stable but converges to a visibly worse answer than the unmodified solver ($\sim 25\times$ worse error against a Sinkhorn reference). The gradient step’s adjoint and the projection’s internal adjoint have to agree on one consistent metric for the linearized-ADMM majorization argument to hold – these pieces are not independently swappable. Both reverts are documented in `prox_nu.py/pipeline_nu.py`’s current docstrings.

9 R ’s weighted adjoint (the FP projection’s remaining piece)

This section is a derivation to check, not a finished result. §8 is why this piece must be right, and consistent with A_t^* , before anything here is implemented again. Unlike §4’s A_t^* , nothing below has been checked against a numerical adjoint identity yet.

9.1 R 's Euclidean transpose, per building block

$R(Q, B) = \mathbf{D}_t Q + B \mathbf{D}_x^T - \varepsilon(\bar{\mathbf{I}}_t Q) \mathbf{L}_x^T$ (nonuniform_grid_xy_variables.tex §8.3–8.4's matrices, Q, B the matrix forms of q, b : $\mathbf{D}_t, \bar{\mathbf{I}}_t$ act by left-multiplication – per space column – and $\mathbf{D}_x, \mathbf{L}_x$ by right-multiplication – per time row, exactly §3's convention). Taking the plain Euclidean (Frobenius) adjoint of each term ($\mathbf{L}_x = \mathbf{L}_x^T$, symmetric):

$$(R^T \Phi)_q = \mathbf{D}_t^T \Phi - \varepsilon \bar{\mathbf{I}}_t^T (\Phi \mathbf{L}_x), \quad (R^T \Phi)_b = \Phi \mathbf{D}_x. \quad (8)$$

This matches `projection_nu.py` as coded today (up to the module's own already-verified sign convention, `deriv_t_at_rho = -D_t^T` – see `validate_nu.py`'s `check_deriv_t_adjoint`):

```
rho_correction = deriv_t_at_rho(phi) + vareps * interp_t_at_rho(deriv_x_at_phi(
    deriv_x_at_m(phi))) # (R^T phi)_q
mx_correction = deriv_x_at_m(phi)
                  # (R^T phi)_b
```

9.2 $R^* = N^{-1} R^T W$, for *any* invertible W – the choice is irrelevant

Lemma 3 (The codomain weight cancels). *For any invertible W (not just $W = I$), $R^* := N^{-1} R^T W$ gives the same projection formula as $W = I$:*

$$v - R^*(RR^*)^{-1}(Rv - d) = v - N^{-1} R^T (RN^{-1} R^T)^{-1}(Rv - d), \quad \forall W.$$

Proof. Write $K := RN^{-1} R^T$. Then $RR^* = RN^{-1} R^T W = KW$, so $(RR^*)^{-1} = (KW)^{-1} = W^{-1} K^{-1}$, and $R^*(RR^*)^{-1} = N^{-1} R^T W \cdot W^{-1} K^{-1} = N^{-1} R^T K^{-1}$ – the W, W^{-1} cancel identically, for any invertible W . $\square$

So R 's codomain (the residual/dual-potential space, where ϕ lives) does not need its own weight assigned at all – not because it *has* none (it could just as well be M , since ϕ lives at the same points ρ does), but because Lemma 3 makes the choice provably immaterial to the projection this computes. Skipping it (equivalently: computing with $W = I$, the cheapest representative of every equally-valid choice) is not an approximation or a simplifying assumption:

$$R^* = N^{-1} R^T, \quad (R^* \Phi)_q = N_q^{-1} [\mathbf{D}_t^T \Phi - \varepsilon \bar{\mathbf{I}}_t^T (\Phi \mathbf{L}_x)], \quad (R^* \Phi)_b = N_b^{-1} [\Phi \mathbf{D}_x], \quad (9)$$

N_q^{-1} applied once to the whole q -correction (not to each piece before combining – linear, so the two give the same answer either way).

9.3 The $N^{-1/2}$ connection

Checking this against the whitening view directly answers “isn't it just $N^{-1/2}$ ”: $\tilde{R} := RN^{-1/2}$, $u := N^{1/2}v$. Solve $\tilde{R} \tilde{R}^T \phi = \tilde{R}u (= Rv, \text{ since } \tilde{R}u = RN^{-1/2} N^{1/2} v = Rv)$, then $x^* = N^{-1/2}(u - \tilde{R}^T \phi) = v - N^{-1} R^T \phi$. So: *one* factor of $N^{-1/2}$ modifies R itself (giving $\tilde{R}$), and it appears *twice* once $\tilde{R} \tilde{R}^T$ is formed – contributing N^{-1} overall, matching (9) exactly. Not two independent $N^{-1/2}$'s; one, appearing on both sides of the assembled system.

9.4 What `_probe_composite_ops` computes today

The module doesn't use (8) in closed form – it probes with unit vectors $v = e_j$ (module docstring: “rather than hand-rederive the closed-form coefficients for a variable `dt_vec...` this module builds the per-mode operator by literal substitution”):

```
drho = ops.deriv_t_at_rho(v)      # (D_t^T v)-piece, sign per the module's convention
irho = ops.interp_t_at_rho(v)    # (I_t^T v)-piece
M0[:,j] = -ops.deriv_t_at_phi(drho, 0, 0)
M1[:,j] = vareps*(ops.deriv_t_at_phi(irho,0,0) - ops.interp_t_at_phi(drho,0,0)) + v[:,0]
M2[:,j] = ops.interp_t_at_phi(irho, 0, 0)
```

Cross-checked against `projection.py`'s closed form (uniform grid, $M0, M1, M2$'s role there): $M0$ is the (q -side, time-only) $\mathbf{D}_t \mathbf{D}_t^T$ contribution ($2/\Delta t^2$ on the diagonal, matching `projection.py`'s `D[1:-1]`); the bare “+ v[:,0]” in $M1$ is the (b -side) $\mathbf{D}_x \mathbf{D}_x^T$ contribution – DCT-diagonalized to a pure scalar eigenvalue $l_{x,k}$ (`projection.py`: plain “+ 1x” in `D[1:-1]`, no other factor), which is why it shows up as bare v (identity in time) rather than any D_t -built stencil – there is no ε or D_t dependence in that specific piece to probe; the rest of $M1$ and all of $M2$ are the diffusion term's self- and cross-interactions.

9.5 Solving the projection: the space eigenvalues survive untouched

(9) gives R^* ; the projection itself needs $R(R^*\Phi)$ ($d = 0$ for R specifically, the FP-feasible set being $\{Rx = 0\}$, `xy_variables.tex` §8.4). Substitute (9)'s $Q^* := (R^*\Phi)_q$, $B^* := (R^*\Phi)_b$ into $R(Q, B) = \mathbf{D}_t Q + B \mathbf{D}_x^T - \varepsilon(\bar{\mathbf{I}}_t Q) \mathbf{L}_x^T$ and expand every term:

$$\mathbf{D}_t Q^* = \underbrace{[\mathbf{D}_t \mathbf{N}_q^{-1} \mathbf{D}_t^T]}_{\text{time-only}} \Phi - \varepsilon \underbrace{[\mathbf{D}_t \mathbf{N}_q^{-1} \bar{\mathbf{I}}_t^T]}_{\text{time-only}} (\Phi \mathbf{L}_x), \quad (10)$$

$$B^* \mathbf{D}_x^T = \underbrace{\mathbf{N}_b^{-1}}_{\text{time-only}} \Phi (\mathbf{D}_x \mathbf{D}_x^T), \quad (11)$$

$$-\varepsilon(\bar{\mathbf{I}}_t Q^*) \mathbf{L}_x^T = -\varepsilon \underbrace{[\bar{\mathbf{I}}_t \mathbf{N}_q^{-1} \mathbf{D}_t^T]}_{\text{time-only}} (\Phi \mathbf{L}_x) + \varepsilon^2 \underbrace{[\bar{\mathbf{I}}_t \mathbf{N}_q^{-1} \bar{\mathbf{I}}_t^T]}_{\text{time-only}} (\Phi \mathbf{L}_x^2). \quad (12)$$

(Using $\mathbf{N}_q^{-1}, \mathbf{N}_b^{-1}$ for the diagonal matrices N_q, N_b act as – §3's left/right-multiplication convention; $\mathbf{L}_x = \mathbf{L}_x^T$.) **Every single term factors as (a time-only matrix, the same at every spatial mode) times Φ , $\Phi \mathbf{L}_x$, or $\Phi \mathbf{L}_x^2$** – and $\mathbf{D}_x \mathbf{D}_x^T, \mathbf{L}_x, \mathbf{L}_x^2$ are exactly the operators the *current, unweighted* code already DCT-diagonalizes, with exactly the same eigenvalues $\lambda_{x,k}, \lambda_{x,k}^2$ it already uses (`problem.lambda_x`) – $\mathbf{N}_q^{-1}, \mathbf{N}_b^{-1}$ never appear inside a space-direction factor, only outside/beside one, because (10)–(12) never multiply a time matrix and a space matrix from the same side of Φ . **This is the “space part stays exactly the same” you asked me to check: it does, term by term, not just plausibly.** After DCT-in- x , mode k 's reduced system is still exactly

$$\tilde{T}_k = \tilde{M}0 + \lambda_{x,k} \tilde{M}1 + \lambda_{x,k}^2 \tilde{M}2, \quad (13)$$

the same quadratic-in- $\lambda_{x,k}$ shape `precomp_banded_proj_nu` already assembles – only the (still exactly tridiagonal, since every piece above is built from compositions of the bidiagonal $\mathbf{D}_t, \bar{\mathbf{I}}_t$) time-direction matrices change:

$$\tilde{M}0 = \mathbf{D}_t \mathbf{N}_q^{-1} \mathbf{D}_t^T, \quad \tilde{M}2 = \bar{\mathbf{I}}_t \mathbf{N}_q^{-1} \bar{\mathbf{I}}_t^T, \quad \tilde{M}1 = \pm \left[\mathbf{D}_t \mathbf{N}_q^{-1} \bar{\mathbf{I}}_t^T + \bar{\mathbf{I}}_t \mathbf{N}_q^{-1} \mathbf{D}_t^T \right] + \mathbf{N}_b^{-1}, \quad (14)$$

$\widetilde{M}0, \widetilde{M}2$'s signs match (10)/(12) directly (both unambiguous); $\widetilde{M}1$'s $\pm$ is deliberately left open here – see below.

Where I got stuck, honestly

I tried to pin down $\widetilde{M}1$'s sign by hand, against the *unweighted* ($\mathbf{N}_q^{-1} \rightarrow I$) case, two ways, and they disagreed: my clean derivation above gives $-\varepsilon[\mathbf{D}_t \bar{\mathbf{I}}_t^T + \bar{\mathbf{I}}_t \mathbf{D}_t^T]$; reading the sign directly off the current code's $M1 = \text{vareps}*(\text{deriv_t_at_phi}(\text{irho}) - \text{interp_t_at_phi}(\text{drho})) + v$ (substituting $\text{drho} = -\mathbf{D}_t^T v$, $\text{irho} = \bar{\mathbf{I}}_t^T v$, the module's established sign convention) gives $+\varepsilon[\dots]$. Chasing the discrepancy further, I found the module's own docstring already flags exactly this kind of trap: `deriv_x_at_phi(deriv_x_at_m(.))` (part of what feeds $\widetilde{M}1$'s λ_x -linear structure) has DCT eigenvalue $-\lambda_{x,k}$, not $+\lambda_{x,k}$ – “verified numerically,” per the module docstring, precisely *because* hand-tracking it is error-prone. This is exactly the situation `_probe_composite_ops` was built to avoid by construction (module docstring: “easy to get a sign or index wrong with no way to self-check the algebra”), and I don't want to guess past it.

What doesn't depend on resolving this: *which* building blocks N_q^{-1}, N_b^{-1} multiply – `drho`, `irho`, and the bare `+v` term, respectively, exactly (14)'s structure – is settled by (10)–(12) regardless of sign, since rescaling an already-correct quantity by a positive diagonal matrix cannot flip which pieces combine. So the code change is still exactly this (unchanged from before):

```
drho = ops.deriv_t_at_rho(v) / dt_dual[:, None]      # N_q^{-1}, once
irho = ops.interp_t_at_rho(v) / dt_dual[:, None]     # N_q^{-1}, once
M0[:,j] = -ops.deriv_t_at_phi(drho, 0, 0)
M1[:,j] = vareps*(ops.deriv_t_at_phi(irho,0,0) - ops.interp_t_at_phi(drho,0,0)) \
          + v[:,0] / dt_vec                          # N_b^{-1}, NOT N_q^{-1}
M2[:,j] = ops.interp_t_at_phi(irho, 0, 0)
```

plus, separately ((9)'s b -block – applied directly where `mx_out` is built, outside `_probe_composite_ops` entirely): `mx_out = psi + dphi_dx / dt_vec[:,None]`. What this section adds is not a sign, but the reason not to hand-derive one: exactly like the rest of this module, the actual $\pm$ in (14) should come from running this code and checking it against R, R^* 's adjoint identity numerically (§9's closing “To verify”), not from further hand algebra.

To verify – before touching projection_nu.py again

1. Does (8)–(9) look right?
2. The proposed fix above – in particular, N_q^{-1} on `drho/irho` but a *different*, N_b^{-1} on $M1$'s bare `+v` term – is that the right split, or is there a piece still missing?
3. Given last time: **before any code changes**, write a numerical adjoint-identity check for R^* against R (mirroring `check_At_star_is_NM_weighted_adjoint`, §4's pattern) – agreed this is the right gate before implementation, given the empirical divergence §8 found last time from skipping exactly that step for the projection?